

Vizuara Research Team

Adaptive Draft-Length Controllers for Speculative Decoding: Spending Speculation Where It Pays

8-Week Research Roadmap for Pramod Kumar Reddy Jella, Vizuara Research Team

June 9, 2026

1 What This Project Is About

Project Snapshot

Target / draft	Llama-3-8B and Qwen2.5-7B targets with EAGLE-3 or a small draft model
Signals	Draft entropy / margin, recent acceptance history, prompt type
Controllers	Entropy threshold (SVIP-style), epsilon-greedy and UCB multi-armed bandit on draft length
Benchmarks	Spec-Bench, MT-Bench, HumanEval, GSM8K
Metrics	mean accepted length, wasted draft tokens, TTFT, TPOT, throughput, net speedup
Big idea	A fixed draft length wastes compute on hard tokens and under-uses easy spans; a cheap controller should set the length per step.
Target	Beat the best single fixed draft length on mixed traffic, across batch sizes and tasks, with no loss of output quality.

Speculative decoding speeds up large-model generation by letting a small **draft** propose several tokens that the large **target** verifies in one parallel pass. A central knob is the **draft length**: how many tokens to propose before stopping to verify. Propose too few and you barely beat plain decoding; propose too many and you waste compute drafting tokens that get rejected. Almost every deployed system picks **one** draft length and applies it to all traffic, which is a compromise that is wrong almost everywhere, because the right length depends on how predictable the next span of text is.

The predictability varies enormously within a single generation. A boilerplate span (a closing bracket, a common phrase) is easy: the draft is confident and a long draft will be almost entirely accepted. A hard token (a surprising fact, the start of a new clause) is uncertain: the draft will likely be rejected after one or two tokens, so a long draft just burns compute. The same variation appears across requests: a code completion behaves differently from an open chat. A **fixed** draft length cannot exploit any of this.

The opening: the draft model already tells you when to stop. Zhang et al. (2024) show that the gap between draft and target can be approximated by the **draft's own prediction entropy**, high entropy means likely rejection, so the draft “knows when to stop.” A controller that reads this cheap signal can lengthen speculation on easy spans and cut it short on hard ones, capturing speedup a fixed length leaves on the table.

This project designs and evaluates a **lightweight, content-aware controller** that sets the draft length per step (or per request) from cheap signals, the draft’s entropy or margin, the recent acceptance history, and the prompt type, and asks whether it beats the best fixed length on mixed production-style traffic. It compares simple, deployable controllers: an entropy threshold in the spirit of SVIP, and bandit controllers (epsilon-greedy and UCB) that treat draft length as an arm to be learned online, as in BanditSpec (Hou et al., 2025). Because speculative decoding with exact verification is lossless, the controller changes *only* speed, never output, so the entire evaluation reduces to one honest question: how much net speedup, and how many wasted draft tokens saved, across tasks and batch sizes, without any quality cost. The result is a practical recipe: which signal, which controller, and how it should adapt as load changes.

Scope & Realism Note

What this means for you: you will work with open models, public benchmarks, and a standard engine, so everything is reproducible. Because exact-verification speculation is lossless, you do not trade accuracy for speed here; you only need to show the controller never changes outputs and to report net speedup with the GPU, batch size, and draft model stated, so the comparison against a tuned fixed length is fair.

2 Research Questions & Timeline

2.1 Central Research Questions

RQ1. Can a lightweight per-step controller that sets the draft length from a cheap signal (draft entropy or margin, recent acceptance history) beat the **best single fixed** draft length on mixed traffic, measured by net speedup and wasted draft tokens, with no change to outputs?

RQ2. Which signal and which controller (entropy threshold vs epsilon-greedy vs UCB bandit) gives the best speedup, and how does the optimal policy shift with batch size and task type (chat, code, math)?

RQ3. How well does a single controller generalise across target/draft pairs and workloads *without* per-workload retuning, and how quickly does an online bandit converge from a cold start?

2.2 Timeline at a Glance

Milestone	Weeks	Key Deliverable(s)
M1: Literature Review & Scoping	1 to 2	Excel matrix (12 papers); fixed-length baseline sweep; controller design; evaluation plan
M2: Controller Implementation & Harness	3 to 4	Entropy + bandit controllers; per-step logging; mixed-traffic harness; EDA notebook
M3: Experiments & Ablations	5 to 6	Controller vs fixed-length results; 5 ablations; failure taxonomy
M4: Analysis, Insights & Manuscript	7 to 8	Speedup curves; convergence analysis; insight report; 6 to 8 page draft manuscript

3 Milestone 1: Literature Review & Scoping (Weeks 1 to 2)

3.1 Objectives

Build grounding across three pillars: (a) speculative decoding and the EAGLE family, especially EAGLE-2’s dynamic draft trees; (b) adaptive-length methods, the entropy stop rule (SVIP) and bandit formulations (BanditSpec); (c) the signals available cheaply at decode time (draft entropy, margin, acceptance history). Run a fixed-length baseline sweep (draft length 1 to 8) to establish the best static length per workload, which is the bar the controller must beat.

3.2 Key Concepts: Draft Length as a Control Problem

The decision “how many tokens to draft now” is a sequential decision under a cheap, noisy signal. It can be framed as a threshold rule (stop drafting when entropy crosses a bound) or as a bandit (each candidate length is an arm; reward is accepted-tokens-per-unit-compute). Both are cheap enough to run inside the decode loop.

Approach	Mechanism	Role in This Project
Fixed length	One draft length for all traffic	Baseline to beat (best static length)
Entropy threshold (SVIP)	Stop drafting when draft entropy is high	Simple, training-free controller
Epsilon-greedy bandit	Explore lengths randomly, exploit the best so far	Online controller, easy to deploy
UCB bandit (BanditSpec)	Upper-confidence-bound choice of length	Principled online controller with low regret

3.3 Reading List (12 Papers)

1. Leviathan et al. (2023). *Fast Inference from Transformers via Speculative Decoding*; ICML 2023.
2. Chen et al. (2023). *Accelerating Large Language Model Decoding with Speculative Sampling*; arXiv.
3. Cai et al. (2024). *Medusa: Simple LLM Inference Acceleration with Multiple Decoding Heads*; ICML 2024.
4. Li et al. (2024). *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*; ICML 2024.
5. Li et al. (2024). *EAGLE-2: Faster Inference of LLMs with Dynamic Draft Trees*; EMNLP 2024.
6. Li et al. (2025). *EAGLE-3: Scaling up Inference Acceleration of LLMs via Training-Time Test*; arXiv.
7. Zhang et al. (2024). *Draft Model Knows When to Stop: A Self-Verification Length Policy (SVIP)*; arXiv.
8. Hou et al. (2025). *BanditSpec: Adaptive Speculative Decoding via Bandit Algorithms*; arXiv.
9. Liu et al. (2025). *Not-a-Bandit: Provably No-Regret Drafter Selection in Speculative Decoding*; arXiv.
10. Chen et al. (2024). *Sequoia: Scalable and Robust Speculative Decoding*; NeurIPS 2024.
11. Kim et al. (2023). *Speculative Decoding with Big Little Decoder*; NeurIPS 2023.
12. Xia et al. (2024). *Unlocking Efficiency in LLM Inference: A Comprehensive Survey of Speculative Decoding (Spec-Bench)*; ACL 2024 Findings.

Core Research Insight

The honest baseline is not “draft length 1” but the **best fixed length, tuned per workload**, because that is what a careful engineer would ship. Beating a tuned fixed length is a real result; beating an untuned one is not. The controller’s advantage must therefore come from *within-stream* variation: it wins by being long on the easy spans and short on the hard ones inside the same generation, which no single static length can do. The project’s core measurement is the gap between the adaptive controller and the per-workload-optimal fixed length, reported with wasted-draft-token counts so the mechanism is visible.

3.4 Evaluation Plan

Metrics: net wall-clock speedup over plain decoding; mean accepted length; wasted draft tokens (drafted but rejected) per accepted token; TTFT, TPOT, throughput; controller overhead. Output equivalence to plain decoding is verified (lossless).

Protocols: (1) Beat-the-best-fixed: compare the controller against every fixed length and against the per-workload-optimal fixed length. (2) Batch sweep: repeat at batch 1, 8, 32, 64. (3) Generalisation:

train/tune on one workload, test on another without retuning.

Baselines: every fixed draft length (1 to 8); the per-workload-optimal fixed length; the SVIP entropy rule; an oracle upper bound that knows the true acceptance for each step.

Deliverable #1

`literature_matrix.xlsx` (12 papers, all columns filled) • 1-page Memo: all 3 RQs, the controller designs (entropy, epsilon-greedy, UCB), and the evaluation plan
 • `notebooks/00_fixed_length_sweep.ipynb`: measure speedup vs fixed draft length per workload and record the best static length.

Acceptance Check: Matrix complete; memo defines all 3 RQs and the controllers; fixed-length sweep identifies the best static length per workload; evaluation plan documented.

4 Milestone 2: Controller Implementation & Harness (Weeks 3 to 4)

4.1 Benchmarks & Workloads

Workload	Size	Description	Use
Spec-Bench	6 domains	Standard speculative-decoding benchmark (translation, QA, RAG, summarisation, math, code).	Primary
MT-Bench	80 multi-turn	Open-ended chat; high within-stream entropy variation.	Chat
HumanEval	164 problems	Python code; long predictable spans.	Code (easy)
GSM8K	1.3K	Math word problems; mixed predictability.	Math
Mixed trace	assembled	Interleaved chat/code/math to emulate production traffic.	Stress

4.2 Tasks

1. **Speculation backbone:** Set up EAGLE-3 (or a small draft) on two targets; expose the draft length as a per-step controllable parameter.
2. **Signals:** Compute cheap per-step signals: draft prediction entropy, top-1 margin, and a running acceptance-rate estimate; verify they correlate with realised acceptance.
3. **Controllers:** Implement (i) an entropy-threshold stop rule, (ii) an epsilon-greedy bandit over candidate lengths, (iii) a UCB bandit; each chooses the next draft length online with negligible overhead.
4. **Mixed-traffic harness:** Build a runner that streams interleaved chat/code/math requests and records per-step length, acceptance, wasted tokens, and timing.
5. **Equivalence check:** Confirm bit-for-bit (or distributional) output equivalence to plain decoding, so any speedup is free.
6. **EDA notebook:** Distribution of optimal length by signal value; acceptance vs entropy; within-stream length variation; controller overhead measurement.

Deliverable #2

`src/controllers/` (entropy, epsilon_greedy, ucb) • `src/serve/` (speculation backbone with per-step length hook) • `src/bench/` (mixed-traffic harness, per-step logging)
 • `notebooks/01_EDA.ipynb`

Acceptance Check: All three controllers run inside the decode loop with low overhead; signals correlate with acceptance; output equivalence confirmed; mixed-traffic harness produces per-step logs; EDA notebook complete.

5 Milestone 3: Experiments & Ablations (Weeks 5 to 6)

5.1 Experiment Matrix

Policy	Adapts?	Primary Metrics
Fixed length (best per workload)	no	speedup, accepted length (baseline)
Entropy threshold (SVIP-style)	per step	speedup, wasted tokens
Epsilon-greedy bandit	online	speedup, regret, convergence
UCB bandit (BanditSpec-style)	online	speedup, regret, convergence
Acceptance-history controller	per request	speedup, wasted tokens
Oracle length (upper bound)	per step	speedup ceiling

5.2 Implementation Details

Controllers: candidate lengths $\{1, 2, 3, 4, 6, 8\}$; entropy threshold tuned on a small dev set; bandits with tuned exploration and a sliding reward window; reward = accepted tokens per unit draft compute. **Measurement:** net speedup and wasted tokens averaged over long mixed streams; wall-clock on one fixed GPU with warm-up; 3 seeds. **Overhead:** measure the controller’s own cost to confirm it is negligible. **Reproducibility:** pinned models, all configs in YAML, one script per table.

5.3 Error Taxonomy

Error Type	Description	Expected Frequency
Over-Drafting	Controller keeps drafting past the rejection point; wasted compute	Higher when the signal lags
Under-Drafting	Controller stops too early on an easy span; speedup left unused	Higher with a conservative threshold
Cold-Start Regret	Online bandit explores poorly before converging	Early in a new stream
Signal Miscalibration	Entropy/margin poorly predicts acceptance for this model	Model-dependent
Batch Interference	Per-request lengths conflict under batched scheduling	Higher at large batch

5.4 Ablation Studies

A1. Signal: entropy vs top-1 margin vs acceptance history vs combination. **A2. Controller:** threshold vs epsilon-greedy vs UCB. **A3. Batch size:** 1, 8, 32, 64; does adaptation still pay under batching? **A4. Candidate set:** coarse $\{1, 4, 8\}$ vs fine $\{1..8\}$ length options. **A5. Generalisation:** tune on one workload, test on others without retuning. Report speedup, wasted tokens, and (for bandits) regret for every cell.

Deliverable #3

`src/run_experiments.py`, `src/evaluate.py` • `results/metrics_all.csv` (every policy: speedup, accepted length, wasted tokens, regret, all seeds) • `results/error_taxonomy.md`
• Ablation tables (A1 to A5) in CSV

Acceptance Check: All cells computed on all workloads; output equivalence re-verified; at least one example per error type; all 5 ablations done; the best controller beats the per-workload-optimal fixed length on mixed traffic.

6 Milestone 4: Analysis, Insights & Manuscript (Weeks 7 to 8)

6.1 Core Visualisations

(1) **Speedup-vs-policy bars** (primary): each controller against the best fixed length and the oracle, per workload. (2) **Length-vs-signal plot**: chosen draft length as a function of entropy/margin, showing the learned policy. (3) **Wasted-token comparison**: drafted-but-rejected tokens per accepted token, per policy. (4) **Bandit convergence**: cumulative speedup (or regret) over a stream from a cold start. (5) **Batch-size curve**: controller advantage as batch grows.

6.2 Insight Report (Answers to All 3 RQs)

I1. State the net speedup of the best controller over the per-workload-optimal fixed length, with wasted-token savings. **I2.** Name the best signal and controller and how the optimum shifts with batch and task. **I3.** Report cross-workload generalisation and bandit convergence speed. **I4.** Quantify controller overhead (must be negligible). **I5.** Practitioner decision table:

Use Case	Recommended Controller	Reasoning
Simple, training-free	Entropy threshold (SVIP-style)	Cheap, no learning, solid gains
Mixed unknown traffic	UCB bandit	Adapts online; low regret
Stable single workload	Tuned fixed length	If traffic never varies, adaptation adds little
Large-batch serving	Per-request acceptance-history	Avoids per-step conflicts under batching
Research upper bound	Oracle length	Shows the speedup still on the table

6.3 Manuscript Outline (6 to 8 pages, Workshop Format)

1. Introduction (fixed-length waste; the control opportunity; RQs; contributions). **2.** Related Work (speculative decoding; dynamic trees; entropy and bandit length control). **3.** Methodology (signals; controllers; equivalence guarantee). **4.** Experimental Setup (targets; workloads; metrics; protocol). **5.** Results (speedup vs fixed and oracle; convergence; ablations). **6.** Discussion (when adaptation pays; signal calibration; batching). **7.** Limitations (engine specificity; signal availability; batch interference). **8.** Conclusion (a deployable adaptive-length recipe; future: joint length-and-draft routing).

6.4 Packaging

Project layout: `adaptive_draft_len/` containing `data/` (workload configs, `data_cards`), `src/` (controllers, `serve`, `bench`, `run_experiments`, `evaluate`, `utils`), `notebooks/` (00 to 02), `results/` (`metrics_all.csv`, `figures`, `error_taxonomy`), `configs/` (YAML), `report/` (LaTeX manuscript), `requirements.txt`, and `README.md`.

Deliverable #4

`results/figures/` (all plots, ≥ 200 dpi) • `results/metrics_all.csv` (all cells + ablations)
 • Insight report (2 to 3 pages, all 3 RQs answered) • `report/manuscript.pdf` (6 to 8 page draft, all sections) • `README.md` + `requirements.txt`

Acceptance Check: All cells populated; speedup, length-vs-signal, and convergence figures generated; output equivalence proven; all 3 RQs answered with numerical evidence; the controller beats the tuned fixed length; third-party clone reproduces the main speedup table within tolerance; manuscript complete (all 8 sections).

7 One-Page Checklist

Project Checklist

W1 to 2 (Discover): ☐ 12 papers summarised in Excel matrix ☐ 3 RQs stated; controllers designed (entropy, epsilon-greedy, UCB) ☐ Fixed-length sweep done; best static length recorded ☐ Signals and metrics chosen ☐ 1-page memo reviewed

W3 to 4 (Build): ☐ Speculation backbone with per-step length hook ☐ Entropy, epsilon-greedy, and UCB controllers implemented ☐ Signals correlate with acceptance ☐ Output equivalence to plain decoding confirmed ☐ Mixed-traffic harness + EDA notebook complete

W5 to 6 (Experiment): ☐ All policy cells computed on all workloads ☐ All 5 ablations (A1 to A5) completed ☐ Failure taxonomy: 30+ cases inspected ☐ Best controller beats the tuned fixed length ☐ Reproducibility test passed (3 seeds)

W7 to 8 (Evaluate): ☐ Speedup, length-vs-signal, and convergence figures generated ☐ Wasted-token comparison produced ☐ Insight report: all 3 RQs answered with evidence ☐ Practitioner decision table populated ☐ Draft manuscript complete (all 8 sections) ☐ Reproducibility pass: third-party clone succeeds

8 Quick Reference Links

- Speculative Decoding (Leviathan et al., 2023)
- EAGLE-2: Dynamic Draft Trees (Li et al., 2024)
- EAGLE-3 (Li et al., 2025)
- SVIP: Self-Verification Length Policy (Zhang et al., 2024)
- BanditSpec (Hou et al., 2025)
- Not-a-Bandit: Drafter Selection (Liu et al., 2025)
- Sequoia (Chen et al., NeurIPS 2024)
- Spec-Bench Survey (Xia et al., 2024)
- EAGLE Official Code (GitHub)
- vLLM Speculative Decoding Docs